

How Speak to Sign works

It translates both ways. **Phase 1** turns spoken English into an Indian Sign Language (ISL) video, built from real signer clips. **Phase 2** watches someone sign into a webcam and turns their signs into written and spoken English. This page covers each step, the tools behind it, what runs where, and what costs money.

2 directions 2,848 sign clips to show 263 signs to recognise 1 paid service (Azure OpenAI)
≤ \$0.002 per sentence

PHASE 1 · SPEECH → SIGN

One sentence, start to finish

A real run of the app. Every stage passes one output to the next, in a fixed order. Only step 2 leaves your machine.

Input audio (4.6 s): "Hello, what is your name? I want to drink water."

STEP 1 · LISTEN

Speech → English text

faster-whisper, base.en model

● Local CPU · free

OUTPUT

Hello, what is your name? I want to drink water.

≈ 3 s



STEP 2 · TRANSLATE

English → ISL gloss

GPT-4.1 on Azure OpenAI, via LangChain + LangGraph

● Cloud API · paid

OUTPUT

HELLO YOUR NAME WHAT I WATER DRINK WANT

≈ 2-4 s

STEP 3 • LOOK UP

Gloss word → video clip

SQLite table gloss → video_path

● Local · free

OUTPUT

8 of 8 words matched →
clips/HELLO.mp4 ...
< 10 ms



STEP 4 • SIGN

Clips → one video

moviepy, falls back to ffmpeg

● Local CPU · free

OUTPUT

isl_...mp4, 18 s, 1280×720, plus a
start and end time for each word
≈ 5-15 s

The timings come from test runs on this Windows laptop. The first request after a start adds about 5 s while Whisper loads. Stitching is the slowest step; setting `VIDEO_BACKEND=ffmpeg` makes it several times faster.

KEY IDEA

What an “ISL gloss” is

ISL is its own language with its own grammar. It is not English spelled out with hands. A **gloss** is the standard way to write signs down: one uppercase English word per sign, in signing order. Converting to gloss is a translation step, and that's why an LLM does it instead of a word-for-word script.

- **Drop articles and “to be”:** a, an, the, is, am, are.
- **Verb at the end** (subject–object–verb).
- **NOT comes after** the verb it negates.
- **Time first**, then topic, then comment.
- **Question words last:** WHAT, WHERE, WHY.
- **No tense endings:** the past is shown with time words or FINISH.

ENGLISH	ISL GLOSS	RULE SHOWN
What is your name?	YOUR NAME WHAT	question word last, “is” dropped
I am going to school tomorrow.	TOMORROW I SCHOOL GO	time first, verb last
I don't understand.	I UNDERSTAND NOT	negation after verb
Have you eaten your food?	YOU FOOD EAT FINISH	past tense via FINISH

ENGLISH	ISL GLOSS	RULE SHOWN
I am so glad to meet you!	I YOU MEET HAPPY	synonym mapped to a sign we have

The prompt has 11 rules and 12 worked examples (“few-shot” prompting). It also lists the signs that have video clips, so the model picks words it can actually show (“glad” → HAPPY). With 2,848 signs, the full list would be too long, so each request sends only the roughly 60 signs related to the sentence. A clean-up step then strips any stray punctuation or leftover “the/is”.

TECH STACK

What we use, and why

LAYER	TECHNOLOGY	WHY THIS CHOICE	RUNS
Language	Python 3.12	Every AI, speech and video library here is Python-first.	● Local
Web server	FastAPI + Uvicorn REST API, file uploads	Small and fast. Request checking and interactive docs at <code>/docs</code> come built in.	● Local
Speech-to-text	faster-whisper OpenAI's open Whisper model, <code>base.en</code> , int8	Runs offline on a normal CPU, is free, and your audio never leaves the machine. It's a faster rebuild of Whisper (CTranslate2) that is about 4x quicker. It's a one-time 150 MB download.	● Local
Translation	GPT-4.1 on Azure OpenAI accessed through the <code>openai</code> SDK	Turning English into ISL gloss means reordering and rewriting the sentence, which is a language task an LLM handles well. Azure gives enterprise billing and keeps data in your Azure tenant.	● Azure

LAYER	TECHNOLOGY	WHY THIS CHOICE	RUNS
LLM framework	LangChain AzureChatOpenAI + LangGraph	The translation runs as a small graph, <i>translate</i> → <i>normalize</i> . That makes it easy to add steps later (a synonym check, a quality check) without rewriting code.	● Local
Sign lookup	SQLite one table: gloss TEXT PRIMARY KEY, video_path TEXT	A single file with nothing to install. Looking up 2,848 rows takes under a millisecond.	● Local
Video	moviepy, with ffmpeg as fallback ffmpeg bundled via imageio-ffmpeg	Joins clips in gloss order and sizes each to 1280x720 at 25 fps. The output plays in any browser (H.264).	● Local
Sign videos	Hugging Face dataset bridgeconn/sign- dictionary-isl	2,848 real ISL signs (3,188 videos), one sign per clip, filmed on green screen. It's free to use under CC BY-SA 4.0.	● Stored locally
Hand tracking	MediaPipe 1.0.1 HandLandmarker + PoseLandmarker (Phase 2)	Google's free on-device tracker. It finds 21 points on each hand and the upper body in real time, even in a browser, and runs the same way in Python for training.	● Browser
Sign recogniser	PyTorch (training) → ONNX Runtime (live) a small Transformer over landmark sequences	PyTorch is the standard for training. ONNX Runtime runs the finished model fast on CPU without PyTorch installed.	● Local

LAYER	TECHNOLOGY	WHY THIS CHOICE	RUNS
Training data	INCLUDE (Zenodo record 4010759)	263 ISL signs with about 16 examples each from Deaf signers, which is enough to learn from. It's free under CC BY 4.0.	● Landmarks stored locally
Speech output	Web Speech API the browser's built-in text-to-speech	It's free, works offline, has no API key, and uses an Indian English voice where available.	● Browser
Frontend	Plain HTML, CSS and JavaScript MediaRecorder for the mic, <code>fetch</code> for the API	No framework and no build step. The API server serves both pages (Speech → Sign and Sign → Speech).	● Browser

DATASET

Where the Phase 1 sign videos come from

ISLV Sign Dictionary by Bridge Connectivity Solutions Pvt. Ltd., published on Hugging Face. It has 2.5 hours of segmented sign videos, one gloss per clip, filmed in 1920x1080 on green screen, plus body-pose data we don't use yet.

The full dataset is about 7 GB, mostly pose data. Our script reads only the file headers over HTTP to build an index of where each word's video sits, then downloads just the videos: **2.1 GB in, 1.1 GB after conversion**, with 0 failures. It's public, so no Hugging Face account or key is needed.

Catch: it comes from a *Bible* dictionary, so it has entries like **PRAY** and **TEMPLE** but lacks everyday words such as PLEASE, SCHOOL, TOMORROW and YESTERDAY. Those can be recorded and added with the import script. The **licence** (CC BY-SA 4.0) means we must credit Bridge Connectivity Solutions, and anything we publish from it must use the same licence.

MISSING WORDS

What happens when a word has no sign

The system never makes up a sign. For each word in the gloss, it tries these in order:

NAME

1 · Play the sign clip

The word is in the database, so its real clip plays.

R · A · H · U · L

2 · Fingerspell it

Names and missing words are spelled with the ISL hand alphabet at 1.5x speed, the way human signers do it. This needs 26 letter clips, which we still have to record.

XYZ

3 · Skip and warn

If neither option works, the word is left out, logged, and shown crossed out in the UI. Nothing crashes.

PHASE 2 · SIGN → SPEECH

From a webcam to a spoken sentence

The reverse direction. The person signs **one word at a time**, lowering their hands between signs. This is a real run from our browser test: a clip of three INCLUDE signs uploaded to the page.

Track hands and body

1

MediaPipe 1.0.1 hand + pose models, running inside the browser

● Browser · free

55 points per frame, 15 frames a second

Split into signs

2

A sign starts when a hand rises above chest level and ends when both hands drop

● Browser · free

3 signs found in a 7.3 s clip

Recognise each sign

3

Our Transformer model, run with ONNX Runtime on the server

● Local CPU · free

I 92% · IT 94% · YOU-PLURAL 91%

Gloss → English

4

GPT-4.1 on Azure OpenAI, a LangGraph node with the reverse of Phase 1's prompt

● Cloud API · paid

"I give it to you all."

Speak it

5

The browser's built-in text-to-speech (an Indian English voice where available)

● Browser · free

spoken audio

Privacy: the camera video never leaves the browser. Only the tracked points (numbers) are sent to the server. Each recognised sign shows its top-5 guesses, so the user can correct a wrong guess before building the sentence.

PHASE 2 · TRAINING

How the recogniser learned

Unlike Phase 1, which only looks clips up, Phase 2 needs a trained model. That model needs many videos of each sign by different people. A dictionary with one video per word isn't enough.

STEP	WHAT WE DID	WHY
Dataset	INCLUDE (AI4Bharat, CC BY 4.0) 263 everyday word signs in 15 categories, about 4,300 videos by Deaf signers, 56.8 GB	It's the main public ISL dataset with about 16 examples of each sign from several signers.
Download	Streamed individual videos out of the 44 zip files with HTTP range requests, 8 at a time. Kept only the tracked points, then deleted each video.	Zenodo gives about 1.3 MB/s per connection, so parallel downloads were 3× faster. The result takes about 30 MB instead of 57 GB.
Same tracker	MediaPipe 1.0.1 with identical <code>.task</code> models, in Python for training and in the browser for live use.	The model sees the same kind of data live as it did in training.
Resting-hand fix	A hand hanging below chest level (more than 1.75 shoulder-widths below the shoulders) is treated as not visible.	INCLUDE films the full body, so resting hands are always visible; webcams usually don't see them. The fix makes both look the same.
Features	Each sign is resized to 32 frames. Each frame has 196 numbers: points relative to the body, hand shape relative to the wrist, and whether each hand is visible. Motion (frame-to-frame change) is added inside the model.	The result doesn't depend on where the person stands, how big they appear, or how fast they sign.

STEP	WHAT WE DID	WHY
Model	A 3-layer Transformer (about 1.8 M parameters), trained on CPU with PyTorch, then exported to ONNX.	It's small and fast (milliseconds per sign), and the server needs only ONNX Runtime, not PyTorch.
Training tricks	Random mirroring (left-handed signers), small rotation and zoom, speed changes and simulated tracking dropouts. Label smoothing, and 120 training passes.	It works for new signers and cameras instead of memorising the training videos.

TRIAL · 9 SIGNS

100%

Pronouns only, 51 test videos. This proved the pipeline worked end to end.

INTERIM · 49 SIGNS

96.2%

Correct first guess on 157 test videos (98.7% within the top 5). Pronouns and Adjectives.

FULL · 263 SIGNS

training...

All 15 categories. The download finishes around 6 pm and training around 7 pm today; this card gets the final score.

These scores come from INCLUDE's official test videos, which the model never saw during training. Accuracy on a new person's webcam will be lower, because lighting, camera angle and signing style all differ.

COST

What you pay for

Only one piece costs money: the Azure OpenAI call. Phase 1 uses it to turn English into gloss, and Phase 2 to turn gloss into English. Everything else, including hand tracking, recognition, speech and one-time training, is open source running on your own machine.

● Paid · pay per token

Azure OpenAI: GPT-4.1

≈ **\$0.90–1.70** per

1,000 sentences

Phase 1: 800 in + 10 out tokens
≈ \$0.0017

Phase 2: 400 in + 12 out tokens
≈ \$0.0009

(GPT-4.1: \$2.00 in / \$8.00 out
per 1M tokens)

With **GPT-4.1-mini** (\$0.40 / \$1.60 per 1M tokens) it's about **\$0.20–0.35 per 1,000 sentences**, which is plenty for this kind of short translation. Token counts were measured on the real prompt; prices are OpenAI's list prices for these models. Azure prices vary by region and deployment type, so check your Azure pricing page.

● Free

Everything else

faster-whisper speech model

\$0 · open source (MIT)

Sign video dataset (Phase 1)

\$0 · CC BY-SA 4.0

INCLUDE training data (Phase 2)

\$0 · CC BY 4.0

MediaPipe, ONNX Runtime, PyTorch

\$0 · open source (Apache / MIT / BSD)

Browser text-to-speech

\$0 · built in

FastAPI, SQLite, moviepy, ffmpeg

\$0 · open source

LangChain / LangGraph

\$0 · open source (MIT)

Compute and one-time training

your own PC

API KEYS

Keys and configuration

There's one key, for Azure OpenAI. It lives in a `.env` file next to the code, never in the source code, and `.gitignore` keeps it out of Git. `.env.example` lists the variable names with empty values.

```
# .env (required)
AZURE_OPENAI_ENDPOINT=https://<your-resource>.openai.azure.com/
AZURE_OPENAI_API_KEY=<key from Azure portal>
AZURE_OPENAI_API_VERSION=2024-10-21
AZURE_OPENAI_DEPLOYMENT_NAME=<your deployment, e.g. gpt-41-isl>

# optional tuning
WHISPER_MODEL=base.en          # tiny.en = faster, small.en = more accurate
VIDEO_BACKEND=auto             # ffmpeg = fastest stitching
FINGERSPELL=1                  # spell missing words once A-Z clips exist
```

Privacy: your voice recording and camera video never leave the computer. Azure only receives short text: the transcribed English (Phase 1) or the recognised sign words (Phase 2). Azure OpenAI doesn't use API data to train models.

Tip: the deployment name is the name *you* chose in Azure AI Foundry, not the model name. Mixing these up is the most common cause of a 404 error.

INTERFACES

API endpoints

The Phase 1 page calls steps 1, 2 and 3–4 separately so it can show live progress; other programs can call the all-in-one endpoint instead. Phase 2 sends one request per recognised sign and one for the sentence.

ENDPOINT	SEND	GET BACK
POST /transcribe	audio file	English text
POST /gloss	English text	ISL gloss words
POST /render	gloss words	MP4 video, plus a start and end time for each word

ENDPOINT	SEND	GET BACK
POST /generate-isl-video	audio file	MP4 video (all four steps in one call)
GET /vocabulary	–	every available sign, plus whether fingerspelling is on
POST /recognize	tracked points for one sign	top-5 guessed signs with confidence (Phase 2)
POST /to-english	list of sign words	a natural English sentence (Phase 2)
GET /sign-model	–	signs the recogniser knows, and its test accuracy

HONEST LIMITS

What it doesn't do (yet)

- **No facial grammar.** Real ISL uses the face and body too, for example raised eyebrows for questions. Joined-up word clips can't show that.
- **Joins between signs aren't smooth.** Each clip starts and ends at rest, so sentences look like a list of words rather than natural signing.
- **Several signers.** Clips from the dataset can switch between people partway through a sentence.
- **Vocabulary gaps.** Everyday words missing from the Bible-based dataset have to be recorded or fingerspelled.
- **No model can invent signs.** Signs are arbitrary like spoken words, so every new sign needs a real source clip.
- **Phase 2 reads one sign at a time.** The signer must pause between words. Continuous, natural signing is still an open research problem.
- **Phase 2 needs a clear view.** Head, shoulders and chest must be in frame, with reasonable light. Accuracy on new people is lower than the test score.
- **The two phases have different vocabularies.** Phase 1 can show 2,848 signs (Bible dictionary); Phase 2 can recognise 263 everyday signs (INCLUDE).
- **Needs Deaf review.** The LLM's gloss and the clip choices should be checked by an ISL signer before any public use.

